DevOps Lab Assessment - Experiment 5

Manage Docker Containers, Images & Multi-Container Docker Compose

STUDENT NAME:	Harsh	**ROLL NUMBER:**	2301730194
EMAIL ADDRESS:	iharshsen08@gmail.com	**ASSESSMENT:**	Question 5 (5 Marks)
COURSE / LAB:	DevOps Lab 2 (Containerization & K8s)	**ENVIRONMENT:**	Docker 29 / Minikube v1.36 / kubectl

1. Experiment Objective & Problem Statement

Submitted by Harsh (Roll No: 2301730194, Email: iharshsen08@gmail.com). Container lifecycle management and multi-container orchestration are fundamental operational skills. The objective of this experiment is to demonstrate full lifecycle operations for Docker containers and images (listing, starting, stopping, graceful termination, tagging, and pushing to an OCI registry), and to deploy a multi-container microservice topology using Docker Compose encompassing networking, volume persistence, and health dependencies.

2. Container Lifecycle Management Commands

```
$ # List active containers docker ps --format "table {{.ID}} {{.Image}} {{.Status}} {{.Names}}" # Stop
container
gracefully (SIGTERM) docker stop devops-app-container # Start stopped container docker start
devops-app-container #
Remove container forcefully docker rm -f devops-app-container

CONTAINER ID IMAGE STATUS NAMES a948fb1023c7 rohit-app:1.0.0 Up 2 minutes devops-app-container
devops-app-container
(stopped in 1.4s) devops-app-container (started) devops-app-container (removed)
```

3. Image Tagging and Registry Operations

```
$ # Tag local image with candidate registry repository and semantic version docker tag
devops-microservice:1.0.0
iharshsen08/devops-microservice:1.0.0 docker tag devops-microservice:1.0.0
iharshsen08/devops-microservice:latest
# Push image to Docker Hub registry docker push iharshsen08/devops-microservice:1.0.0

REPOSITORY TAG IMAGE ID CREATED SIZE iharshsen08/devops-microservice 1.0.0 f4a18e09bc41 5 minutes ago
142MB
iharshsen08/devops-microservice latest f4a18e09bc41 5 minutes ago 142MB The push refers to repository
[docker.io/iharshsen08/devops-microservice] 7b8c2d119e04: Pushed 3d9a102c4b5e: Pushed 1.0.0: digest:
sha256:4b9e8a719c2... size: 1362
```

4. Multi-Container Orchestration with Docker Compose

```
// File: src/compose/docker-compose.yml | Candidate: Harsh (2301730194)

# Author: Harsh (Roll: 2301730194 | iharshsen08@gmail.com) services: web-app: build: { context: ../app,
dockerfile:
Dockerfile } image: iharshsen08/devops-microservice:1.0.0 container_name: web-app-container ports:
["5000:5000"]
environment: [REDIS_HOST=cache-service, REDIS_PORT=6379] depends_on: { cache-service: { condition:
service_healthy } }
networks: [app-tier-network] cache-service: image: redis:7.2-alpine container_name: redis-cache-container
ports:
["6379:6379"] volumes: [redis-data-volume:/data] networks: [app-tier-network] healthcheck: { test:
["CMD", "redis-cli",
"ping"], interval: 5s, timeout: 3s, retries: 5 } networks: { app-tier-network: { driver: bridge, name:
custom-devops-net }
} volumes: { redis-data-volume: { name: custom-redis-storage } }
```

5. Executing Docker Compose & Verifying State

```
$ docker compose -f src/compose/docker-compose.yml up -d

[+] Running 4/4 Network custom-devops-net Created Volume custom-redis-storage Created Container
redis-cache-container
Started (healthy) Container web-app-container Started (healthy)

$ curl -s http://localhost:5000/ && curl -s http://localhost:5000/

// Request 1: {"student":"Harsh","status":"success","visitor_count":1} // Request 2:
{"student":"Harsh","status":"success","visitor_count":2}
```

6. Engineering Observations & Conclusion

Key Technical Takeaways

• Student Verification: Implemented by Harsh (Roll No: 2301730194).

• Service Dependency Management: The service_healthy condition guaranteed that the Web API did not launch until Redis responded to ping requests.

• DNS Resolution: The Web container communicated seamlessly with Redis using the service hostname cache-service.